

Computer Vision Project - Group 11

Thomas Pagano

Arham Muse Hassan

Florent Volvert

1 Problem Statement

For the 3 tricks we have common 2 sub-problems which are

- How can we detect and keep track of the selected objects? What we will call the object trajectories
- How can we detect the wand and keep track of the wand ? The wand trajectory.

Each tricks have their own sub-problems

- Trick 1: how do we spot the cloak, when do we consider that the cloak is covering an object, and how do we create the invisible effect on the cloak?
- Trick 2: When we manage to obtain the wand and object positions, we need to determine when the wand touches an object and how to apply the desired effect to the object.
- Trick 3: One of the main difficulties comes from inaccuracies in the wand trajectory. In particular, sudden vertical jumps caused by tracking errors can lead to unrealistic movements of the ball, which required additional filtering and error detection mechanisms.

We wanted to make a solution that does not rely on ArUco markers.

2 Methodology

Let's divide the methodology for each sub-problem.

2.1 Tracking of the objects

The starting point is the detections of an ML model, which is YOLO. This model can recognize objects in a given frame and return bounding boxes and labels for the different detected objects.

The first issue is that the model does not detect all objects correctly every time; we cannot assume that YOLO will detect our object in each frame. The identification is limited and not consistent, and we cannot rely on labels alone. If we have two balls, labels alone are not sufficient to discriminate between the objects.

Our solution is to use a Kalman filter. Each object is represented by a Kalman filter that is fed by YOLO detections, where we assume that each YOLO detection corresponds to one of our targeted objects and assign the detection to the Kalman filter that corresponds the most to it by using the DIoU metric.

Since we cannot discriminate the objects using YOLO, the first detection for each object is done manually by us.

To make this method more robust, we filter out the detection labels of unwanted objects using a "black-list" in our code, which is created by looking at all the YOLO detections in the video. When a Kalman filter gets no detection for too long, we put it in sleep mode (stop predicting) so that it does not make unrealistic predictions.

All these predictions of positions and box sizes are stored in a .txt file. To fill the prediction gaps, we use linear interpolation so that we obtain a continuous trajectory.

2.2 Tracking of the wand

The process is really similar to object tracking. The difference comes from the detector.

For the wand, we detect the red color and try to isolate “blobs.” Blobs are groups of pixels within a given size range, used to discriminate “a small red thing”, like the one on the wand.

2.3 Trick 1

To detect the cloak, we assume that it is green and detect the green pixels. To filter out the “stray green pixels,” we apply an erosion followed by a dilation.

We consider that an object is covered when a (and only one) green component overlaps the “object kernel.” The object kernel is a sub-box of the object box, which makes potential overlaps easier.

By doing so, we can extract the overlapping component (which we hope is the cloak) alone and apply invisibility only to this component, so that other green objects are not affected.

Invisibility is simply created by the `cv2.inpaint()` function.

2.4 Trick 2

To detect whenever the wand is touching an object, we can check when the bounding box of the wand overlaps with the bounding box of an object.

To make the object change color, we play within the HSV scale and try to keep the same color contrast.

Pour les 2 autres je te laisse arham(pas besoin d’être long)

2.5 Trick 3

The goal of Trick 3 is to create an illusion of telekinesis, where the ball appears to move under the influence of the wand without any physical contact.

This trick relies on pre-processing trajectories of both the ball and the wand. Instead of directly repositioning the ball, the movement of the wand controls the velocity of the ball, resulting in a smoother and more natural motion.

At each frame, the displacement of the wand is computed and smoothed using an exponential moving average to reduce noise. However, the wand trajectory may contain occasional errors, especially sud-

den vertical jumps caused by incorrect detections. These abnormal variations are detected by comparing the current vertical motion with the previous one. When such a jump is detected, the influence of the wand is strongly reduced for that frame, preventing unrealistic movements of the ball.

3 Results

The results are quite good; further polishing of the effect could lead to a better visual impression.

We tried to make the code as tunable as possible to adapt to specific videos, which is beneficial, but it also means that the algorithm is roughly semi-automatic.

But still, there are some things that we could hardly handle. For example, if YOLO does not detect a targeted object at all, the object tracking gets kind of stuck (we still have a completely manual backup strategy). The same goes for the wand: having too much red similar to the wand could be a major obstacle.

For example, trick 3 in our static video suffers significantly from the red tracking.

4 Contributions

Thomas PAGANO contributed to: Trick1, object/wand trajectory, and most of the code reorganization

Arham MUSE HASSAN contributed to: Mainly Trick 2 and object/wand trajectory

Your FLORENT VOLVERT contributed to: Mainly Trick 3.

5 AI Utilization

As computer vision rookies, we used AI—more precisely ChatGPT—as a reliable first partner to ask questions and explore ideas. It helped us visualize what certain functions do, how to process video data, and better understand key course concepts such as YOLO and Kalman filtering. We also frequently

sought ideas, particularly for designing the third effect in Trick 2. Of course, all ideas and concepts were ultimately implemented and adapted by us for this project.

There are still some small functions, like the cIoU metric, that were mainly made by ChatGPT, of course, we took the time to understand what was done and test it.

We also use it to correct our language errors in this report.

6 GitHub Link

`https://github.com/Thopag/Computer_Vision/
tree/main`